



Vazquez Rangel Guillermo Larzen

2024171016

IDGS17

Gerardo Ramírez Villarreal

Seguridad en el Desarrollo de Aplicaciones

Objetivo

Configurar Promtail para enriquecer los logs de VulnerableApp con labels, mandarlos a Loki y comprobar que se vean igual en Seq y en Grafana.

Paso 1. Verificación de la infraestructura existente

Se confirmó con `docker ps` que grafana, loki y promtail (de la práctica P4H-1) seguían corriendo, que Loki seguía como Data Source en Grafana, y que VulnerableApp seguía generando logs en consola, carpeta Logs y Seq.

Container CPU usage 4.79% / 1600% (16 CPUs available)

Container memory usage 1.41GB / 3.42GB

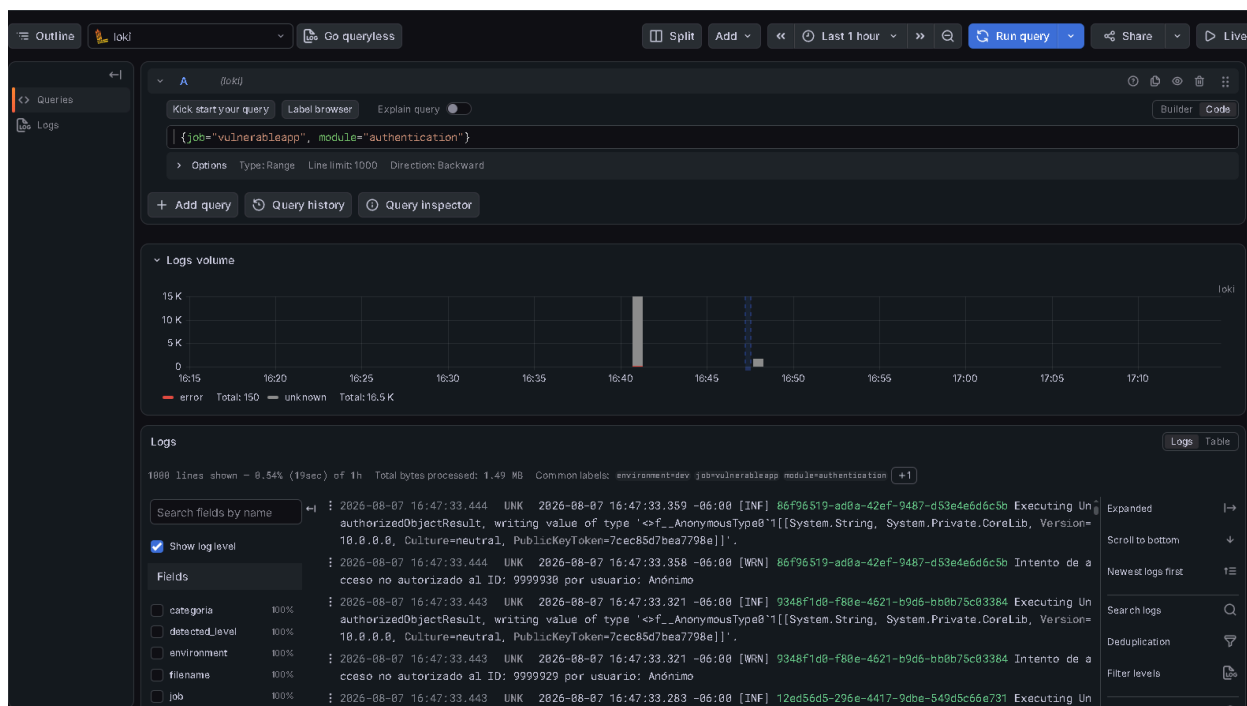
Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
☐	seq	86b39df6a886	datastus/seq:5341.5341	Show all ports (2)	0.68%	214.4MB / 3.5GB	5.98%	283MB / 7.35MB	5.44MB / 457KB	67	59 minutes	🔗 🔍 ⋮ 🗑️
☐	nul_pointer_b	-	-	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	1 day ago	🔗 🔍 ⋮ 🗑️
☐	vulnerableapp	-	-	-	4.11%	1.2GB / 21.02GB	34.16%	5.06GB / 153.55f	5.17MB / 6.51ME	30, 1, 22B, 22, 23	53 minutes	🔗 🔍 ⋮ 🗑️
☐	vulnerableapp	109ee8225cd8	vulnerable	-	0.01%	56.18MB / 3.5GB	1.57%	207MB / 5.45MB	1.65KB / 126B	30	59 minutes	🔗 🔍 ⋮ 🗑️
☐	zap	d4dcb9e0096e	zapproxy/zap	-	0%	14.1MB / 3.5GB	0.39%	48.4MB / 8.19KB	1.2KB / 126B	1	59 minutes	🔗 🔍 ⋮ 🗑️
☐	pentest_mysql	973eb70014ca	mysql/mysql	-	1.34%	837.3MB / 3.5GB	23.34%	2.57GB / 74MB	1.95KB / 126B	228	59 minutes	🔗 🔍 ⋮ 🗑️
☐	loki	e2e0332451ca	grafana/loki:3100.3100	-	1.02%	102.8MB / 3.5GB	2.87%	636MB / 55.8MB	557KB / 3.9MB	22	59 minutes	🔗 🔍 ⋮ 🗑️
☐	grafana	ea3d32169f44	grafana/grafana:3000.3000	-	0.84%	190MB / 3.5GB	5.3%	1.58GB / 17MB	4.62MB / 2.61ME	23	59 minutes	🔗 🔍 ⋮ 🗑️
☐	promtail	9c07cf58bab8	grafana/promtail	-	0.9%	24.76MB / 3.5GB	0.69%	36.6MB / 1.29ME	914B / 126B	17	53 minutes	🔗 🔍 ⋮ 🗑️
☐	sonarqube-infra	-	-	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	0, 0	59 minutes	🔗 🔍 ⋮ 🗑️
☐	sonarqube	87a42dcecdca	sonarqube	9000.9000	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	59 minutes	🔗 🔍 ⋮ 🗑️

Showing 12 items



Paso 2. Análisis del archivo promtail-config.yml

Antes de modificar el archivo se analizó la función de cada sección, tal como fue configurada en la práctica anterior:

Sección	Función
server	Los puertos donde Promtail se pone a escuchar. El <code>http_listen_port</code> saca las métricas y el estado de salud, mientras que el <code>grpc_listen_port</code> en cero apaga esa función porque aquí no hace falta.
clients	A dónde va a mandar Promtail los registros ya ordenados y etiquetados. Aquí apunta directo a Loki usando el nombre del servicio dentro de la red de Docker.
positions	El archivo donde Promtail anota por qué línea o byte va leyendo cada documento. Así, si se apaga el contenedor, no pierde su lugar ni lee las cosas dos veces.
scrape_configs	La lista de tareas para recolectar información. Cada una define qué archivos revisar, qué etiquetas fijas ponerle y cómo transformar los datos.
static_configs	Dentro de la tarea anterior, define sin rodeos qué archivos buscar y las etiquetas principales, incluyendo la ruta exacta que Promtail tiene que vigilar.
labels	Las etiquetas que sirven para organizar los registros. Son clave porque Loki busca primero por estas etiquetas y ya después revisa el texto dentro de ese grupo.

Paso 3. Configuración del monitoreo de múltiples archivos

Serilog genera un archivo por día (log-20260703.txt, etc.), así que `__path__`: `/var/log/VulnerableApp/*.txt` ya cubre todos sin tocar la config; Promtail detecta archivos nuevos solo.

Para separar familias de archivos a futuro (ej. security-*.txt) se puede usar más de un job en `scrape_configs`, cada uno con su propio `__path__` y labels:

`scrape_configs`:

- `job_name`: `vulnerable_app_logs`

`static_configs`:

- `targets`: `[localhost]`

`labels`:

`job`: `vulnerableapp`

`__path__`: `/var/log/VulnerableApp/log-*.txt`

- `job_name`: `vulnerable_app_security_logs`

`static_configs`:

- `targets`: `[localhost]`

`labels`:

`job`: `vulnerableapp`

`categoria`: `security`

`__path__`: `/var/log/VulnerableApp/security-*.txt`

Promtail necesita la ruta física porque, a diferencia de Seq (que recibe push de Serilog), funciona haciendo tailing de archivos: lee directo del disco. Por eso hace falta el bind mount `./Logs:/var/log/VulnerableApp:ro` y un `__path__` correcto; sin eso no encuentra nada que leer.

Paso 4. Enriquecimiento de registros mediante labels

Las labels son metadatos clave-valor que Promtail agrega a cada línea antes de enviarla a Loki. A diferencia de una propiedad de Serilog, la label sí forma parte del índice y define el stream.

Para separar Application, Security y Audit (que llegan revueltos en el mismo archivo) se usa un `pipeline_stage` de regex que detecta la palabra clave y la promueve a label:

pipeline_stages:

- regex:

```
expression: "(?i)(?P<categoria>Security|Audit)"
```

- labels:

categoria:

Si la línea no trae ninguna palabra clave, la label categoria queda vacía y se toma como Application. Esto es más eficiente que buscar texto: primero se filtra por label (barato) y luego se puede filtrar texto o agregar (count over time, rate) sin depender de la palabra exacta.

Paso 5. Aplicación de la configuración

Se reinició solo el servicio promtail (docker compose restart promtail) y se revisó docker logs promtail para confirmar que no hubiera errores de conexión con Loki.

Container CPU usage

4.79% / 1600% (16 CPUs available)

Container memory usage

1.41GB / 3.42GB

Show charts

Q Search

Only show running containers

		Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
		seq	86b39df6a8b6	datastudio	5341:5341	0.68%	214.4MB / 3.5GB	5.98%	28.3MB / 7.35MB	5.44MB / 457KB	67	59 minutes	
		null_pointer_b	-	-	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	1 day ago	
		vulnerableapp	-	-	-	4.11%	1.2GB / 21.02GB	34.16%	5.06GB / 153.55B	5.17MB / 6.51MB	30, 1, 22B, 22, 23	53 minutes	
		vulnerable_el	109ee8225cd8	vulnerable	-	0.01%	56.18MB / 3.5GB	1.57%	20.7MB / 5.45MB	1.65KB / 126B	30	59 minutes	
		zap	d4dc9e4096e	zapproxy/z	-	0%	14.1MB / 3.5GB	0.39%	48.4MB / 8.19KB	1.2KB / 126B	1	59 minutes	
		pentest_mss	973e870014ca	mysql/serv	-	1.34%	837.3MB / 3.5GB	23.34%	2.57GB / 74MB	1.95KB / 126B	228	59 minutes	
		loki	e2e0332451ca	grafana/lo	3100:3100	1.02%	102.8MB / 3.5GB	2.87%	636MB / 55.8MB	557KB / 3.9MB	22	59 minutes	
		grafana	ea3d32169f44	grafana/gr	3000:3000	0.84%	190MB / 3.5GB	5.3%	1.58GB / 17MB	4.62MB / 2.61MB	23	59 minutes	
		promtail	9c07cf58bab8	grafana/pr	-	0.9%	24.76MB / 3.5GB	0.69%	36.6MB / 1.29MB	914B / 126B	17	53 minutes	
		sonarqube-inf	-	-	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	0, 0	59 minutes	
		sonarqube	87a42decdda	sonarqube	9000:9000	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	59 minutes	

Showing 12 items

Paso 6. Generación de actividad en VulnerableApp

Se corrió VulnerableApp generando logins correctos/incorrectos, CRUD, warnings y excepciones para tener eventos de las tres categorías. Se confirmó que seguían llegando a consola, Logs y Seq.

The screenshot displays the Seq application interface, showing a log stream of events from VulnerableApp. The logs are organized into a table with columns for time, message, and source. The messages include HTTP requests, endpoint executions, and error messages. The right sidebar shows a filter for 'Errors'.

Log Stream (Left Panel):

- 07 Aug 2026 16:47:34.932 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 9.4391ms
- 07 Aug 2026 16:47:34.931 HTTP GET /Home/Error responded 200 in 5.4979 ms
- 07 Aug 2026 16:47:34.930 Executed endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.929 Executed action VulnerableApp.Controllers.HomeController.Error (VulnerableApp) in 2.7725ms
- 07 Aug 2026 16:47:34.928 Executed ViewResult - view Error executed in 1.026ms.
- 07 Aug 2026 16:47:34.927 Executing ViewResult - running view Error.
- 07 Aug 2026 16:47:34.927 Error detected. RequestId: 00-caf020ad9ea323b6f77823b38a0759a-f8d4429545fc3a60-00
- 07 Aug 2026 16:47:34.925 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller VulnerableApp.Controllers.HomeController (...)
- 07 Aug 2026 16:47:34.925 Executing endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.923 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
- 07 Aug 2026 16:47:34.891 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 9.1576ms
- 07 Aug 2026 16:47:34.890 HTTP GET /Home/Error responded 200 in 5.2499 ms
- 07 Aug 2026 16:47:34.889 Executed endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.889 Executed action VulnerableApp.Controllers.HomeController.Error (VulnerableApp) in 2.5953ms
- 07 Aug 2026 16:47:34.889 Executed ViewResult - view Error executed in 0.9079ms.
- 07 Aug 2026 16:47:34.887 Executing ViewResult - running view Error.
- 07 Aug 2026 16:47:34.886 Error detected. RequestId: 00-469c95a4795072a8fa2c385d9f9b265-96d189a5a8233d04-00
- 07 Aug 2026 16:47:34.885 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller VulnerableApp.Controllers.HomeController (...)
- 07 Aug 2026 16:47:34.884 Executing endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.882 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
- 07 Aug 2026 16:47:34.882 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 11.9214ms
- 07 Aug 2026 16:47:34.848 HTTP GET /Home/Error responded 200 in 5.4151 ms
- 07 Aug 2026 16:47:34.847 Executed endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.847 Executed action VulnerableApp.Controllers.HomeController.Error (VulnerableApp) in 2.7955ms
- 07 Aug 2026 16:47:34.846 Executed ViewResult - view Error executed in 0.9585ms.
- 07 Aug 2026 16:47:34.845 Executing ViewResult - running view Error.
- 07 Aug 2026 16:47:34.844 Error detected. RequestId: 00-3872a1d777e8bcb00ffbf57a00da001b-1025d1591a01eb3b-00
- 07 Aug 2026 16:47:34.843 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller VulnerableApp.Controllers.HomeController (...)
- 07 Aug 2026 16:47:34.842 Executing endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.840 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null
- 07 Aug 2026 16:47:34.811 Request finished HTTP/1.1 GET http://localhost:5018/Home/Error - 200 null text/html; charset=utf-8 8.8208ms
- 07 Aug 2026 16:47:34.810 HTTP GET /Home/Error responded 200 in 5.8035 ms
- 07 Aug 2026 16:47:34.809 Executed endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.808 Executed action VulnerableApp.Controllers.HomeController.Error (VulnerableApp) in 2.4416ms
- 07 Aug 2026 16:47:34.807 Executed ViewResult - view Error executed in 0.8523ms.
- 07 Aug 2026 16:47:34.807 Executing ViewResult - running view Error.
- 07 Aug 2026 16:47:34.806 Error detected. RequestId: 00-0ba45b9a2e73e07111775387041c3a0-6472114734d9db9-00
- 07 Aug 2026 16:47:34.805 Route matched with {action = "Error", controller = "Home"}. Executing controller action with signature Microsoft.AspNetCore.Mvc.IActionResult Error() on controller VulnerableApp.Controllers.HomeController (...)
- 07 Aug 2026 16:47:34.804 Executing endpoint VulnerableApp.Controllers.HomeController.Error (VulnerableApp)
- 07 Aug 2026 16:47:34.802 Request starting HTTP/1.1 GET http://localhost:5018/Home/Error - null null

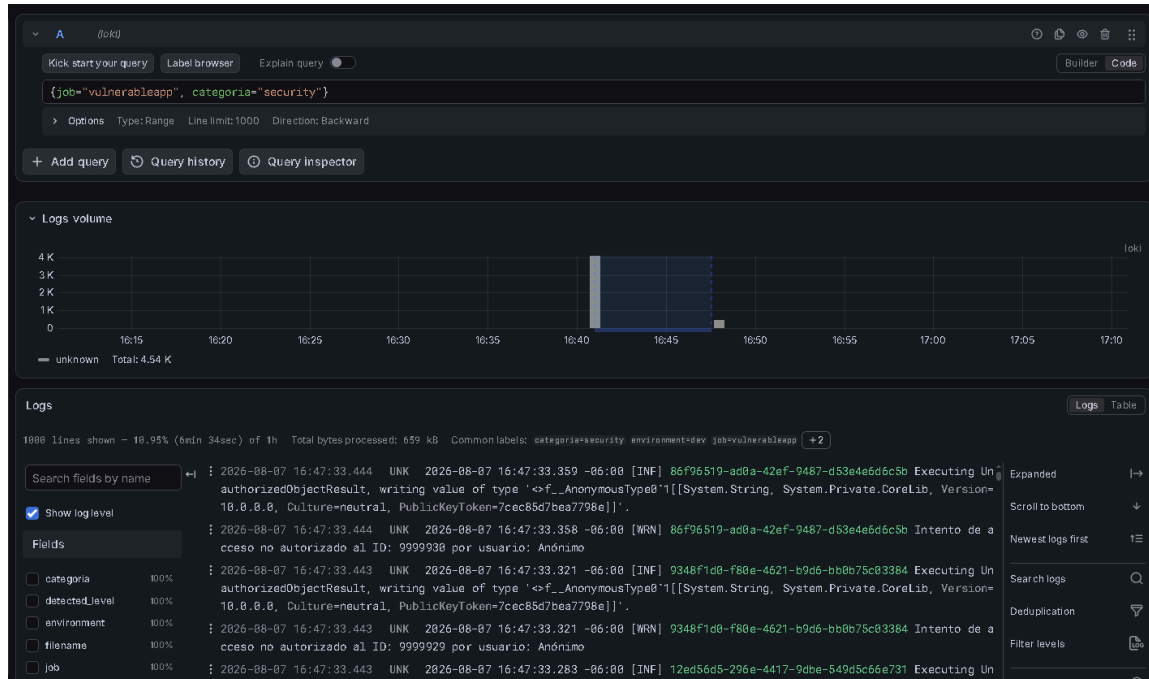
Right Panel (Signals):

- None
- Level
- Errors
- Warnings
- Exceptions
- + 2 more

Bottom Bar: 2025.2.16.202 Individual License

Paso 7. Validación de los registros desde Grafana

Desde Explore (Loki) se corrieron `{job="vulnerableapp"}`, `{job="vulnerableapp", categoria="security"}` y `{job="vulnerableapp", categoria="audit"}`, confirmando que coinciden con lo visto en Seq.



Paso 8. Comparación de resultados

Se analizaron los mismos eventos en ambas herramientas, identificando diferencias en búsqueda, filtrado, organización y capacidades de análisis:

Característica	Seq	Grafana + Loki
Búsqueda de texto	Revisa todo el mensaje y las propiedades estructuradas usando un lenguaje propio parecido a SQL.	LogQL busca primero por etiquetas y luego filtra el texto dentro de ese grupo (<code> =</code> , <code>~</code>), sin indexar todo el contenido.
Filtros por etiquetas	No usa etiquetas; se filtra directo por las propiedades internas de Serilog (<code>Level</code> , <code>CorrelationId</code>).	Las etiquetas (<code>job</code> , <code>categoria</code>) son la base para filtrar y forman el índice principal de Loki.
Paneles visuales	Pantallas diseñadas para una sola aplicación de desarrollo y enfocadas en corregir errores.	Paneles totalmente personalizados que juntan varias fuentes distintas (Loki, Prometheus, etc.) en un solo lugar.
Consultas avanzadas	Consultas tipo SQL sobre los datos estructurados que manda Serilog.	LogQL permite hacer sumas, conteos y cálculos de velocidad (<code>count_over_time</code> , <code>rate</code> , <code>sum by</code>) sobre los grupos etiquetados.
Crecimiento y tamaño	Pensado para uso local o para una sola copia	Separa el almacenamiento del índice por etiquetas, lo que deja crecer el sistema para manejar

	del programa mientras se está programando.	muchos servicios y volúmenes enormes de datos.
Uso principal	Ideal para revisar a detalle y solucionar problemas durante el desarrollo de un programa.	Observabilidad general para múltiples servicios ya listos en un entorno real o de producción.

Pregunta de reflexión

¿Qué ocurriría si Promtail no almacenara la posición del último registro leído en positions.yaml?

Sin positions.yaml, Promtail perdería la referencia de por dónde iba leyendo cada vez que se reinicia. Según el comportamiento por defecto, o reenvía líneas ya procesadas (duplicados, conteos distorsionados) o se salta lo generado mientras estuvo apagado (eventos perdidos). En ambos casos se rompe la confiabilidad de la línea de tiempo para investigar incidentes.

Reto

Se agregaron las labels environment=dev, module=authentication y module=orders. Como authentication y orders conviven en el mismo archivo, module también se saca con un pipeline_stage de regex:

scrape_configs:

- job_name: vulnerable_app_logs

static_configs:

- targets: [localhost]

labels:

job: vulnerableapp

environment: dev

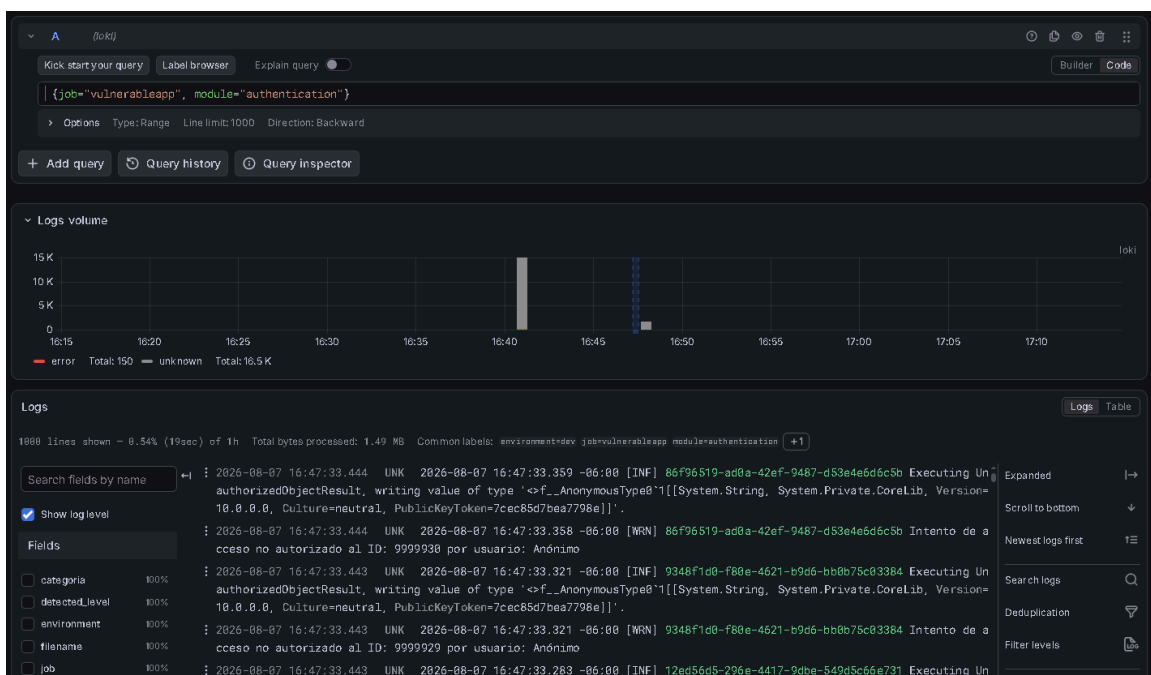
__path__: /var/log/VulnerableApp/*.txt

pipeline_stages:

- regex:
expression: "(?i)(?P<categoria>Security|Audit)"
- labels:
categoria:
- regex:
expression: "(?i)(?P<module>Authentication|Login|Order)"
- labels:
module:

Pasos que seguimos en Grafana para filtrar solo el módulo de autenticación:

1. Procedimiento realizado en Grafana para filtrar únicamente el módulo Authentication:
2. • 1. Reiniciar el servicio promtail para aplicar la nueva label module.
3. • 2. Generar tráfico de autenticación en VulnerableApp (login correcto e incorrecto).
4. • 3. Abrir Grafana → Explore → seleccionar Loki como Data Source.
5. • 4. Ejecutar la consulta `{job="vulnerableapp", environment="dev", module="authentication"}`.
6. • 5. Verificar que solo aparecen líneas de autenticación y comparar el conteo contra Seq.



Conclusiones

Se confirmó que enriquecer logs con Promtail no requiere tocar VulnerableApp: basta un `pipeline_stage` de regex para derivar labels (categoria, module) del contenido que ya emite Serilog. Seq y Grafana+Loki resultan complementarios: Seq es más ágil para depurar en desarrollo, mientras que Grafana+Loki escala mejor para observabilidad centralizada en producción.